

MolmoAct2 · arXiv 2605.02881 · Ai2 / allenai

Empiricist

11+3 · all seats empty

Think on/off + Hz/latency

55.8 Act2 / 12.7 Think @ H100 H=10

Oct 28 paper club · VLA Architectures · quiet build

Spine: geometric S2 $\neq$ semantic HL $\neq$ stage voting · FM motor

FALSE: “has $\pi_{0.5}$ HL” / “has RTC” / “has BEHAVIOR S2 voting”

CRITICAL SPINE — three outer loops (do NOT conflate)

All three call an FM chunk expert (System 1 / motor). Different intermediate discrete states.

MolmoAct2 Think

Adaptive depth / geometry

- 10×10 depth codes (100)
- VQ depth lattice
- Regen if RGB cos < 0.996
- Cached static cells
- Geometric System-2

NOT semantic subtasks

$\pi_{0.5}$ HL

Semantic subtask language

- AR subtask strings
- Optional boxes
- Task language hierarchy
- Then FM LL chunk
- Semantic System-2

NOT depth lattice

BEHAVIOR'25 S2

Stage IDs + voting

- Task-progress stages
- Classifier + voting
- Non-Markov memory
- Then FM expert
- Checklist System-2

NOT scene geometry

FALSE claims: “has $\pi_{0.5}$ HL” · “has RTC” · “has BEHAVIOR S2 voting”

TENTATIVE: BEHAVIOR may want depth + stage complementary · RTC absent — latency via FM cache + CUDA Graphs + Think reuse

Empiricist hypotheses H1-H5

H1

Think vs flat

Think > flat on LIBERO-Long matched FT; gap shrinks on saturated suites

H2

VS $\pi_{0.5}$

Released MolmoAct2 ordinarily beats public $\pi_{0.5}$ on matched LIBERO/DROID proxy

H3

KV depth

Per-layer KV > hidden-state after short matched FT (Table 11 direction)

H4

Latency honesty

Think SR gain $\neq$ free: amortized Hz $\ll$ flat (12.7 vs 55.8)

H5

S2 family

Depth $\neq$ HL $\neq$ stage; combining stage/HL + flat may beat Think alone on long-horizon

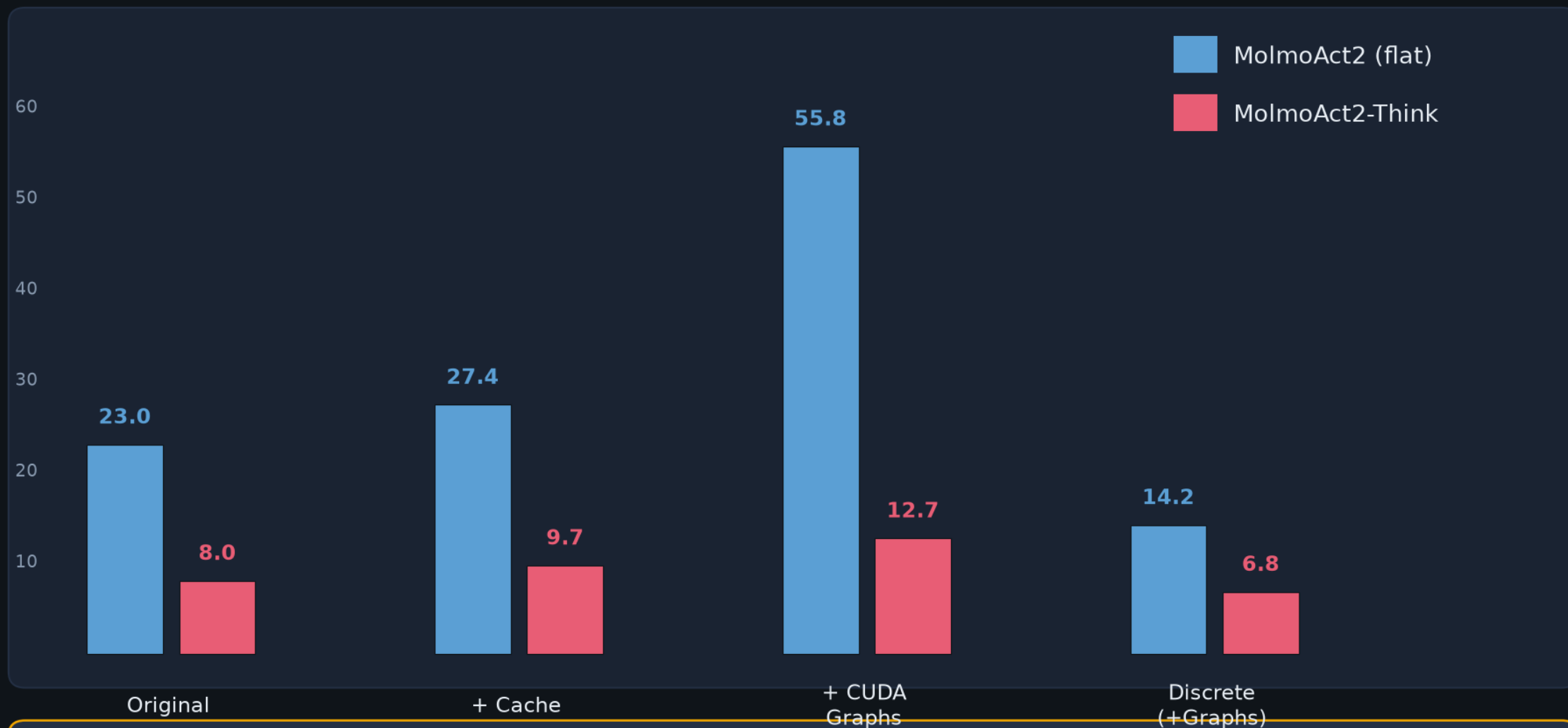
Experiment matrix — P0 / P1 (small-scale)

P	Experiment	Measure	Smoke
P0	Think vs flat continuous (same ckpt)	SR + ms/chunk + Hz	Y
P0	vs $\pi_{0.5}$ on LIBERO / DROID proxy	Ordinal SR matched	Y/P
P0	Think vs flat on LIBERO-Long	Long Δ + suite SR	Y
P1	Per-layer KV vs hidden	Avg SR (95.9 vs 94.0)	Partial
P1	K flow samples {1,2,4,8}	Avg SR	Partial
P1	Adaptive depth ablate (gate/noise)	SR + latency	Partial
P1	$H \in \{8,10,15,30\} \times \text{OL vs RH}$	SR vs Hz	Y

Defer: full 64×H100 pre/post · 720 h YAM retrain · 8-task 50-trial real suite

Inference control rate — Think on/off + optimizations

$$H=10 \cdot 1 \times H100 \cdot \text{LIBERO} \cdot \text{Hz} = H / \text{end-to-end chunk latency}$$



RTC absent — latency via FM cache + CUDA Graphs + Think reuse · don't quote SR without Hz

KPI tattoo — must-quote numbers

LIBERO avg

97.2%

Think 98.1%
 $\pi_{0.5}$ 96.9%

LIBERO Long

93.2→95.4

Think headroom
(ceilings elsewhere)

MolmoSpaces

37.7

vs $\pi_{0.5}$ 34.5
(+3.2)

DROID OOD

87.1%

vs $\pi_{0.5}$ 45.2
real 15-trial

SO-100

56.7%

vs π_0 45.3

YAM 8-task

~50%

vs OFT 35.5

ER VLM

63.8%

9/13 best

Hz Act2/Think

55.8 / 12.7

H100 Graphs

GPU honesty — quote SR with control tags

Required log tuple beside every success number

(policy, think/HL/stage, H, open-loop|RH, NFE, Hz, ms, GPU, VRAM)

Never CPU-fallback as “GPU run” · exit if #SBATCH --gres=gpu but CUDA false

Eval flat LIBERO

1× 16-40 GB

hours

Eval Think

1× 20-48 GB

hours (slower)

Latency / NFE sweep

1× 16-48 GB

hours

Short FT LoRA

1× 40-80 GB

1-4 days

Paper pre+post

64× H100

OUT OF SCOPE

Env outlines: /workspace/molmoact2-quiet/env_outline/ · no jobs in quiet run

Control checklist — Empiricist / Visionary bridge

1. Quote SR only with (policy, think/HL/stage, H, OL|RH, NFE, Hz, ms, GPU)
2. Do not treat Think Hz as “solved real-time” without Graph/cache flags
3. Prefer Comet-style receding when porting to BEHAVIOR (paper is open-loop)
4. If testing “System 2,” pick ONE ontology per ablation cell
5. Don’t cargo-cult open-loop on long-horizon household tasks
6. RTC absent in MolmoAct2 — treat wrap as follow-on experiment (TENTATIVE)

Ablation highlights (LIBERO)

Backbone discrete Long

Molmo2 77.6 → Molmo2-ER 83.6

+6.0

Conditioning

hidden 94.0 → per-head 94.8 → per-layer 95.9

KV wins

Flow samples K

K=1 94.15 → K=8 95.90

monotone-ish

Fine-tune design

expert-only 93.05 · full+discrete 97.20

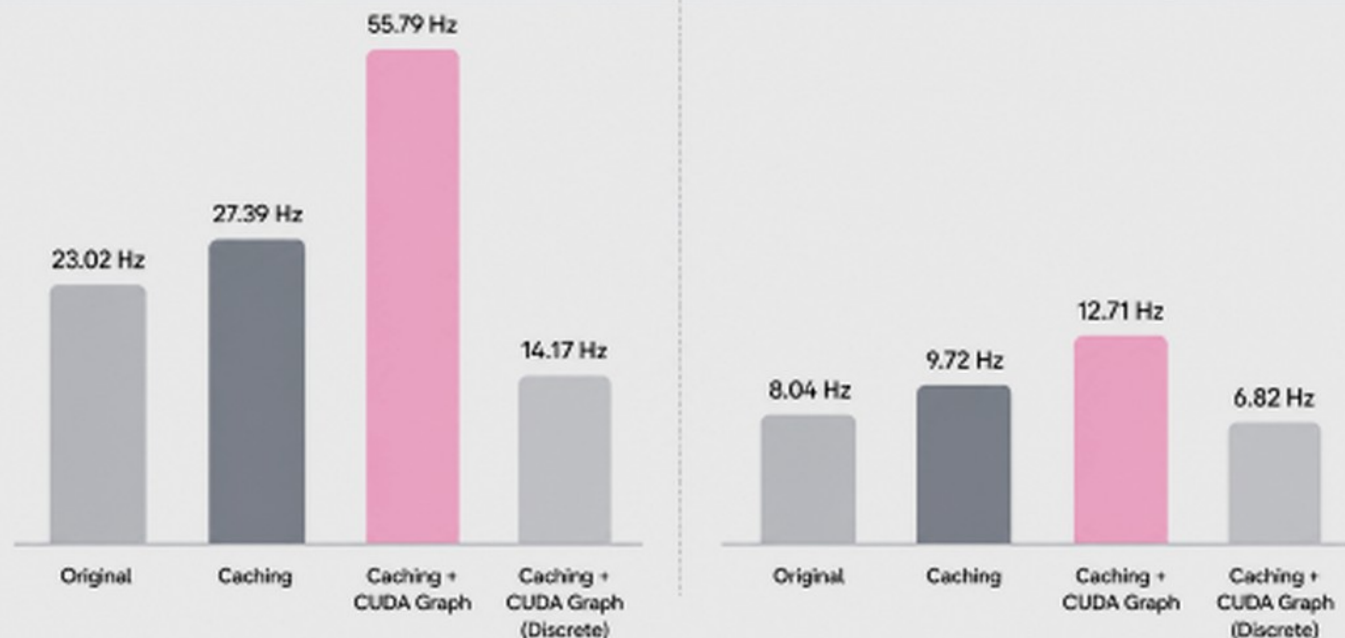
co-train

Think recipe

full mixed+noise+gate → 98.10

gates matter

Fig.8 · Control rate vs cache / CUDA Graphs



Measured on H100 with action horizon 10 on LIBERO.

Figure 8 Inference control rate after caching and CUDA Graph optimizations. Control rate is action horizon divided by end-to-end action-generation latency; measurements use horizon 10 on LIBERO with a single H100.

MOLMOACT2 also exposes a discrete action path, in which the VLM autoregressively decodes action tokens for the entire action chunk. We measured this path under the same optimized graph as above. MOLMOACT2 runs at 14.17 Hz and MOLMOACT2-THINK at 6.82 Hz i.e., 3.94× and 1.86× slower than the corresponding continuous path. This gap comes from the large VLM decoding overhead, while the action expert emits the entire chunk in a small fixed number of flow matching steps through a smaller cross-attention head, with the VLM cache reused across steps. We therefore use the continuous path as the default deployment option.

Table 8 · LIBERO suite rates

Baseline	Spatial	Object	Goal	Long	Average
TraceVLA (Zheng et al., 2024)	84.6%	85.2%	75.1%	54.1%	74.8%
OpenVLA (Kim et al., 2024)	84.7%	88.4%	79.2%	53.7%	76.5%
SpatialVLA (Qu et al., 2025)	88.2%	89.9%	78.6%	55.5%	78.1%
CoT-VLA (Zhao et al., 2025)	87.5%	91.6%	87.6%	69.0%	83.9%
π_0 (Black et al., 2024)	96.8%	98.8%	95.8%	85.2%	94.2%
ThinkAct (Huang et al., 2025)	88.3%	91.4%	87.1%	70.9%	84.4%
MolmoAct-7B-D (Lee et al., 2025)	87.0%	95.4%	87.6%	77.2%	86.6%
GR00T N1.7 (NVIDIA et al., 2025)	97.7%	97.5%	<u>98.5%</u>	<u>94.4%</u>	97.0%
$\pi_{0.5}$ (Intelligence et al., 2025)	<u>98.8%</u>	98.2%	98.0%	92.4%	96.9%
NORA-1.5 (Hung et al., 2025)	97.3%	96.4%	94.5%	89.6%	94.5%
MOLMOACT2	97.8%	100.0%	97.8%	93.2%	<u>97.2%</u>
MOLMOACT2-THINK	98.8%	<u>99.8%</u>	98.5%	95.4%	98.1%

unseen during training. In all fine-tuning experiments, we initialize from the MOLMOACT2-POST checkpoint and fine-tune on the benchmark-specific training data.

In simulation, we evaluate on two benchmarks. RoboEval (Wang et al., 2025b) comprises a bimanual Franka Emika Panda setup with human-expert demonstrations across 8 tasks requiring bimanual coordination. LIBERO (Liu et al., 2023), following prior work (Kim et al., 2024), consists of four task suites—LIBERO-Spatial, LIBERO-Object, LIBERO-Goal, and LIBERO-Long—each containing 500 demonstrations across 10 tasks. We fully fine-tune MOLMOACT2 and compare it against state-of-the-art VLAs trained under similar protocols, including TraceVLA (Zheng et al., 2024), OpenVLA (Kim et al., 2024), SpatialVLA (Qu et al., 2025), CoT-VLA (Zhao et al., 2025), π_0 (Black et al., 2024), ThinkAct (Huang et al., 2025), MolmoAct (Lee

Fig.6 · RoboEval success + quality

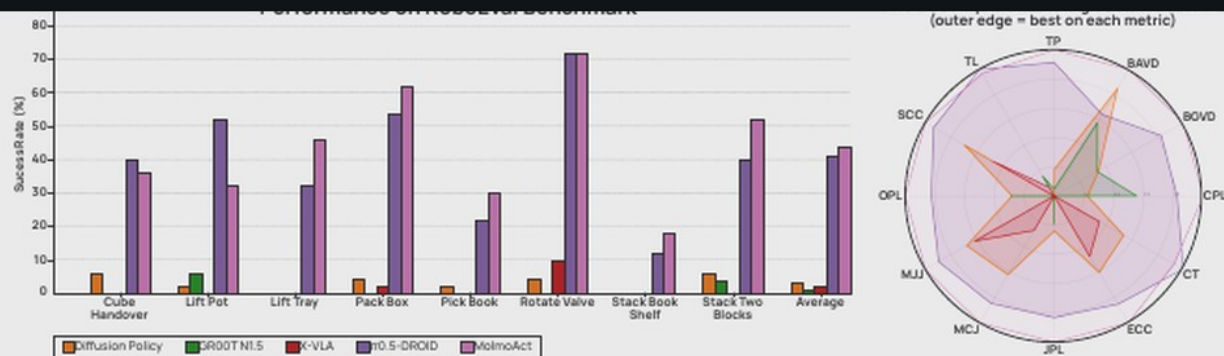


Figure 6 Performance comparison on the RoboEval benchmark. (A) Task-wise success rates (%) across eight manipulation tasks for Diffusion Policy, GR00T N1.5, X-VLA, $\pi_{0.5}$ -DROID, and MolmoAct. MolmoAct achieves the highest or near-highest success rates on most tasks, with particularly strong performance on long-horizon tasks such as *Pack Box* and *Rotate Valve*. $\pi_{0.5}$ -DROID is the strongest baseline overall, while other methods exhibit lower and more variable performance across tasks. (B) Radar plot of normalized performance across behavioral and outcome metrics, averaged over all tasks (outer edge indicates best performance per metric). MolmoAct consistently dominates across nearly all metrics, indicating improvements not only in task success but also in efficiency, stability, and trajectory quality. *Abbreviations:* CT = completion time, TL = trajectory length, JPL = joint path length, CPL = Cartesian path length, CJ = Cartesian jerk, JJ = joint jerk, SC = self-collisions, SL = slip count.

In the real world, we test MOLMOACT2-DROID and MOLMOACT2-SO100/101 on their respective pretrained embodiment setups under challenging out-of-distribution conditions: camera poses are randomly initialized without conforming to any dataset visual distribution, using two cameras (wrist and a single allocentric view); all objects are unseen during training; and the environments themselves are out-of-distribution relative to the training data. For the DROID embodiment, we compare MOLMOACT2-DROID against $\pi_{0.5}$ -DROID (Intelligence et al., 2025), fine-tuned on the DROID dataset, and MolmoBot (Deshpande et al., 2026), fine-tuned on the large-scale MolmoSpaces dataset. We evaluate on five tasks ranging from single- to multi-object pick-and-place: *apple_on_plate*, *pipette_in_tray*, *red_cube_in_tape_roll*, *knife_in_box*, and *objects_in_bowl*. For the SO-100/101 setup, we use the SO-100 robot, retaining the wrist camera and adding a third-person external camera with a randomly initialized position. We compare MOLMOACT2-SO100/101 against SmolVLA (Shukor et al., 2025) and π_0 -SO100/101, a variant of π_0 that we fine-tuned on MOLMOACT2-SO100/101 DATASET. Both baselines are pretrained or fine-tuned at scale on the SO-100/101 dataset. We evaluate five pick-and-place tasks with novel objects and randomly initialized camera poses. Every real-world policy is assessed over 15 trials, with partial credit awarded for near-successful executions. Further details are provided in the Appendix.

Results. MOLMOACT2-DROID achieves strong zero-shot performance on all DROID-style setups, in both simulation and the real world. In simulation, MOLMOACT2-DROID is the state-of-the-art VLA model across MolmoSpaces and MolmoBot, outperforming all baselines on both benchmarks. $\pi_{0.5}$ -DROID is the runner-up on each, with MOLMOACT2 achieving an absolute gain of +10.6% on MolmoBot and +3.2% on MolmoSpaces,

Empiricist · close

P0: Think vs flat $\times \pi_{0.5}$ proxy · always log Hz + ms/chunk.

Tattoo: 97.2/98.1 · 37.7 · 87.1 · KV 95.9>94.0 · 55.79/12.71 Hz

Q&A · Oct 28 · MolmoAct2 / 2605.02881

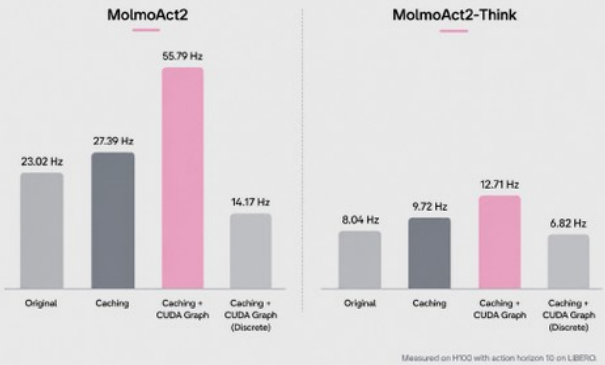


Figure 8 Inference control rate after caching and CUDA Graph optimizations. Control rate is action horizon divided by end-to-end action-generation latency; measurements use horizon 10 on LIBERO with a single H100.

MOLMOACT2 also exposes a discrete action path, in which the VLM autoregressively decodes action tokens for the entire action chunk. We measured this path under the same optimized graph as above. MOLMOACT2 runs at 14.17 Hz and MOLMOACT2-THINK at 6.82 Hz i.e., 3.94× and 1.86× slower than the corresponding continuous path. This gap comes from the large VLM decoding overhead, while the action expert emits the entire chunk in a small fixed number of flow matching steps through a smaller cross-attention head, with the VLM cache reused across steps. We therefore use the continuous path as the default deployment option.

7 Related work

Generalist robot manipulation policies. In recent years, robotic manipulation research has shifted focus from task-specific designs toward developing generalist policies capable of operating across diverse scenarios and embodiments (Berscheid et al., 2019; Brohan et al., 2022; Dasari et al., 2019; Ebert et al., 2021; Fang et al., 2023; Jang et al., 2022; Khazatsky et al., 2024; Mandlekar et al., 2018; Walke et al., 2023; Shafiuallah et al., 2023). A representative approach is the development of vision-language-action (VLA) models. In this paradigm, vision-language models (VLMs)—pretrained on vast, web-scale multimodal data (e.g., image, video, text)—are further trained on collection of diverse robotics datasets (Black et al., 2024; Brohan et al., 2022; Zheng et al., 2024; Zhao et al., 2025; Team et al., 2025b; Li et al., 2024b; Qu et al., 2025; Kim et al., 2024). This knowledge transfer significantly reduces the reliance on domain-specific robotics data, which is typically expensive to collect via teleoperation. However, with the notable exception of our previous work, MolmoAct (Lee et al., 2025), reproducing frontier VLA models (e.g., the π series (Black et al., 2024; Intelligence et al., 2025)) remains highly challenging due to closed-source training data and code. MOLMOACT represents our latest effort to democratize VLA research by releasing full training code, data, and model checkpoints that achieve state-of-the-art performance across a wide range of robotic manipulation benchmarks.

Embodied reasoning for robotic manipulation. While textual chain-of-thought (CoT) prompting (Wei et al., 2022) has advanced reasoning in domains like program synthesis and mathematics, its benefits for robotic manipulation remain limited, likely because tacit physical knowledge is rarely articulated in the text corpora used to train backbone LLMs. This has motivated a growing body of work on non-textual ‘embodied CoT’ that leverages visual or spatial representations, including object bounding boxes and points (Zawalski et al., 2024; Yuan et al., 2024; Li et al., 2025), future image predictions (Zhao et al., 2025), latent reasoning (Tur